

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Цифровая аналитика
«Генератор карт местности»

Выполнил студент группы 5130201/40003

Цомаева Д.Б.

Проверил

Мулюха В.А.

« ____ » _____ 2026 г.

г.Санкт-Петербург, 2026 г.

1 Техническое задание

1.1 Создание материков

Пользователь должен нарисовать замкнутую линию, отражающую примерные границы материка.

Программа по этой линии рисует береговые границы по алгоритму midpoint displacement.

1. Начинаем с отрезка с заданными конечными точками
2. Находим середину отрезка
3. Смещаем ее по вертикали на случайную величину
4. Повторяем рекурсивно для каждого нового отрезка

В конце получаем замкнутый контур материка и заливаем его зеленым.

1.2 Создание карты высот

1. Пользователь рисует примерные границы литосферных плит и задает их движение.
2. Если плиты движутся друг к другу программа рисует горы.
3. В ином случае впадины.
4. Есть высота, является уровнем моря, все, что ниже ее красится в оттенки синего.
5. Все, что выше в оттенки зеленого.

Для более реалистичного перехода будем использовать карты шумов.

1.3 Создание карты влажности

Для начала в программе есть карта ветров описанная по следующим правилам

1. Пассаты (тропики): Постоянные ветры, дующие от тропиков к экватору. В Северном полушарии они северо-восточные, в Южном – юго-восточные.
2. Западные ветры (умеренные широты): Преобладают в средних широтах. В Северном полушарии имеют юго-западное направление, в Южном – северо-западное.

3. Восточные (полярные) ветры: Дуют от полюсов к умеренным широтам. В Северном полушарии – северо-восточные, в Южном – юго-восточные.

Позволяем пользователю настроить скорость ветров.

Создаем также карту полос освещенности:

1. Жаркий пояс (тропический): Расположен между Северным и Южным тропиками.
2. Умеренные пояса (северный и южный): Находятся между тропиками и полярными кругами.
3. Полярные пояса (холодные, северный и южный): Расположены от полярных кругов до полюсов.

Пользователь задает среднюю глобальную температуру на планете.

На основе этого и высот создаем карту температур.

Таким образом карта влажности создается с учетом распределения ветров (если ветер с моря дует в горы, то за горой будет сухо, а перед – влажно), а также близости точки к воде и высоты точки.

1.4 Биомы

На основе всех построенных карт создаем биомы по следующим правилам.

1. Влажно.
 - (a) Очень жарко: тропический лес.
 - (b) Тепло: лес.
 - (c) Холодно: тайга.
 - (d) Очень холодно: тундра
2. Сухо.
 - (a) Очень жарко: пустыня.
 - (b) Тепло: саванна.
 - (c) Холодно: степь.
 - (d) Очень холодно: тундра

1.5 Интерфейс

Интерфейс включает в себя следующие части:

1. Поле.
2. Кнопки.

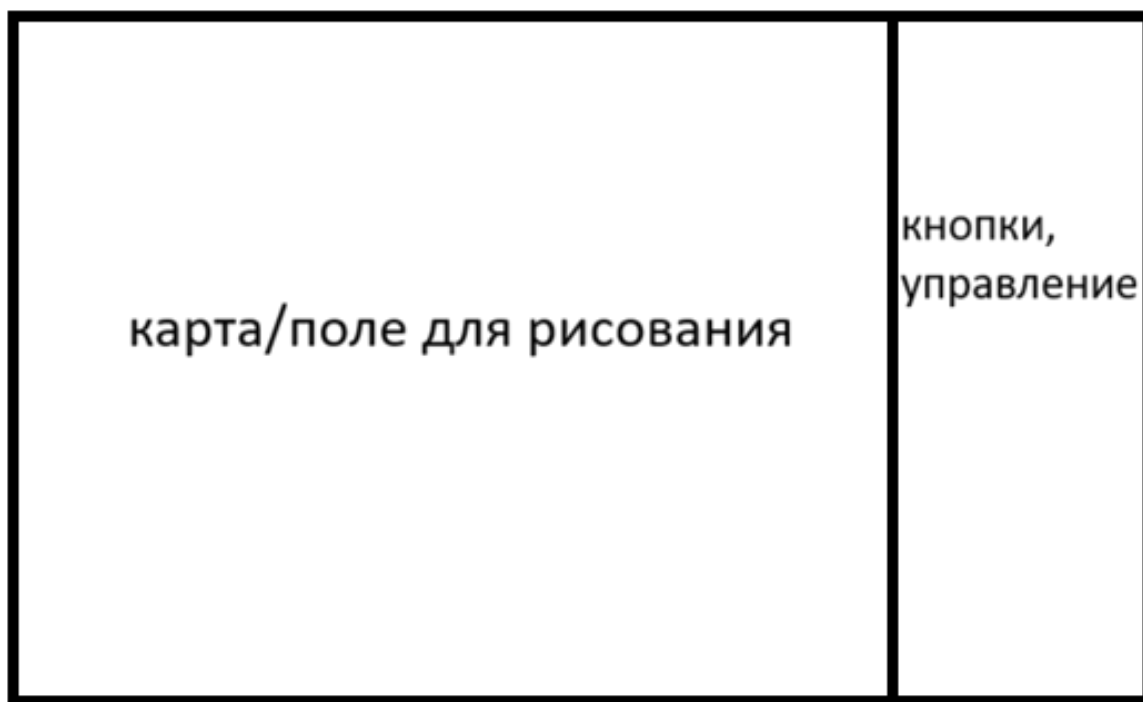


Рис. 1. Пример интерфейса.

Пользователь будет рисовать в поле очертания материков, литосферных плит. Нажимая на кнопки сбоку он будет давать программе команды по генерированию и настройке генерирования, удалению, сохранению результата.

Для создания проекта используется язык python.

1.5.1 Отображение карт

Все карты, которые создает программа будут показываться отдельно, поверх основной карты и будут полупрозрачными.

Отображение осуществляется следующим образом:

1. Влажность: от желтого (сухо) к зеленому (влажно).
2. Точная скорость ветров обозначается в меню справа (формат: название пояса – скорость) , а направление стрелочками на карте (длина пропорциональна скорости).

3. Биомы также будут визуализироваться по цветам, как влажность и карта высот.

1.5.2 Проекция карты

Карта является плоской. Для имитации сферы будут осуществлены следующие правила:

1. Правый и левый край карты будут соединены (т.е. если литосферная плита выйдет за границу слева, то она окажется в правой части карты).
2. Так как карта уже разделена на пояса, сделаем для каждого пояса коэффициент искажения (тем больше, чем дальше от экватора).
3. Будем использовать в подсчетах влажности и температур этот коэффициент.

1.5.3 Смена сезонов и времени суток

Смена суток/сезонов учитываться не будет, реализуется статичная карта.

2 История промтов

Для выполнения задания использовалась нейронная сеть DeepSeek.

Было использовано три чата, два из которых представляют большой объем запросов и ответов нейросети. Для удобства ниже представлены ссылки на диалоги.

Чат 1: <https://chat.deepseek.com/share/1qmsggy6x32xu7ro9z>

Чат 2: <https://chat.deepseek.com/share/r066vkvyb63oay0u7t>

Чат 3: <https://chat.deepseek.com/share/q6ydu8eqpyb14i82gg>

3 Описание работы программы

Программа представляет собой генератор процедурных карт местности с учётом тектоники литосферных плит, ветровой циркуляции, температурных поясов и влажности. Реализована на языке Python с использованием библиотек PyQt5 (графический интерфейс) и NumPy (матричные вычисления). Взаимодействие с пользователем осуществляется через графическое окно с панелью управления и интерактивным полем для рисования.

3.1 Алгоритм генерации

Работа программы разбита на последовательные этапы:

1. Создание материка. Пользователь рисует замкнутый контур. Программа применяет алгоритм `midpoint displacement` (рекурсивное смещение середины отрезка) для фрактального сглаживания береговой линии.
2. Задание литосферных плит. Пользователь рисует 2–3 плиты, задаёт направление и скорость их движения. При столкновении плит в зоне границы формируются горы (высота пропорциональна скорости сближения), при расхождении – впадины.
3. Генерация карты высот. Наложение тектонических эффектов, шума Перлина и хаотического шума. Учитывается коэффициент широтного искажения: на экваторе горы выше и рельеф контрастнее, у полюсов – сглажен. Уровень моря настраивается ползунком.
4. Генерация карты влажности. Сначала создаётся карта ветров (пассаты, западные ветры, полярные) с настраиваемой скоростью. Затем вычисляется температурная карта (зависит от широты, глобальной температуры и высоты). Влажность рассчитывается с учётом расстояния до воды, высоты, температуры, эффекта дождевой тени и широтного искажения.
5. Генерация биомов. На основе трёх параметров (высота, температура, влажность) по таблице решений определяется тип биома: тропический лес, пустыня, тайга, тундра, горы, океан и др.

3.2 Структура проекта

Проект организован в виде одного файла `main.py` (или `terrain_generator.py`), содержащего все модули:

- `MapCanvas` – класс холста для рисования материков и плит, отображения карт.
- `TerrainGeneratorApp` – главное окно с элементами управления и обработкой событий.
- `BiomeType(Enum)` – перечисление типов биомов.
- `Plate` – класс, хранящий границы плиты, вектор и скорость движения.

Вспомогательные функции реализуют:

- алгоритм `midpoint displacement` (`displace_segment`);

- упрощение контура Дагласа–Пекера (`douglas_peucker`);
- шум Перлина и хаотический шум;
- гауссово размытие матриц высот и влажности;
- проверку принадлежности точки полигону (с учётом цилиндрической проекции – склейка левого и правого краёв).

3.3 Особенности реализации

- Цилиндрическая проекция: левая и правая границы карты соединены.
- Широтное искажение: функция `get_distortion_factor` возвращает коэффициент от 1.0 (экватор) до 0.4 (полюса), который умножается на высоту гор и влажность – на полюсах рельеф сглажен и климат суше.
- Эффект дождевой тени: при расчёте влажности трассируется путь против ветра; если ветер идёт с океана и встречает горы – влажность перед горами повышается, за горами падает.
- Визуализация: пользователь может переключать отображение между картой высот, картой влажности, картой биомов и ветрами (стрелки с длиной, пропорциональной скорости). Предусмотрено сохранение карты в `json` и загрузка из него.

3.4 Результат

На выходе программа формирует трёхслойную карту (высоты → влажность → биомы), полностью определяемую пользовательскими настройками: формой материков, тектоникой плит, скоростью ветров и глобальной температурой. Все расчёты учитывают имитацию сферичности через широтное искажение и склейку краёв.

4 Заключение

В ходе выполнения работы разработано программное приложение на языке Python с использованием библиотек PyQt5 и NumPy, реализующее генератор процедурных карт местности. Программа позволяет пользователю создавать материки с фрактальной береговой линией, задавать движение литосферных плит для формирования гор и впадин, а также генерировать карты высот, влажности и биомов с учётом ветровой циркуляции, температурных поясов и широтного искажения. Реализована цилиндрическая проекция со склейкой краёв и возможность сохранения/загрузки результатов в формате JSON.

5 Приложение. Код программы

```
1     import sys
2 import math
3 import random
4 import json
5 import numpy as np
6 from enum import Enum
7 from collections import defaultdict
8 from PyQt5.QtCore import QByteArray
9
10 from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
11                               QHBoxLayout, QGridLayout, QLabel, QPushButton,
12                               QFrame,
13                               QScrollArea, QSlider, QSpinBox, QGroupBox,
14                               QMessageBox, QFileDialog)
15 from PyQt5.QtCore import Qt, QPoint, QRect, QPointF, QRectF
16 from PyQt5.QtGui import QPainter, QColor, QPen, QBrush, QPolygonF,
17     QPainterPath, QImage
18
19 # ----- Биомы и плиты -----
20 class BiomeType(Enum):
21     TROPICAL_RAINFOREST = "Тропический лес"
22     FOREST = "Лес"
23     TAIGA = "Тайга"
24     TUNDRA = "Тундра"
25     DESERT = "Пустыня"
26     SAVANNA = "Саванна"
27     STEPPE = "Степь"
28     OCEAN = "Океан"
29     SEA = "Море"
30     MOUNTAIN = "Горы"
31
32     def to_dict(self):
33         return self.value
34
35     @classmethod
36     def from_dict(cls, value):
37         for member in cls:
38             if member.value == value:
39                 return member
40         return cls.OCEAN
41
42 class Plate:
43     def __init__(self, points, direction, speed, direction_name):
44         self.points = points
45         self.direction = direction
46         self.speed = speed
47         self.direction_name = direction_name
48
49     def to_dict(self):
```

```

48         return {
49             'points': [(p.x(), p.y()) for p in self.points],
50             'direction': self.direction,
51             'speed': self.speed,
52             'direction_name': self.direction_name
53         }
54
55     @classmethod
56     def from_dict(cls, data):
57         points = [QPointF(x, y) for x, y in data['points']]
58         return cls(points, tuple(data['direction']), data['speed'], data['
direction_name'])
59
60 # ----- Вспомогательные функции -----
61
62 def point_in_polygon(x, y, polygon):
63     if not polygon or len(polygon) < 3:
64         return False
65     inside = False
66     n = len(polygon)
67     for i in range(n):
68         x1, y1 = polygon[i].x(), polygon[i].y()
69         x2, y2 = polygon[(i+1) % n].x(), polygon[(i+1) % n].y()
70         if ((y1 > y) != (y2 > y)) and (x < (x2 - x1) * (y - y1) / (y2 - y1) +
x1):
71             inside = not inside
72     return inside
73
74 def polygon_to_path(polygon):
75     if not polygon or len(polygon) < 3:
76         return None
77     path = QPainterPath()
78     path.addPolygon(QPolygonF(polygon))
79     return path
80
81 # ----- Холст -----
82 class MapCanvas(QWidget):
83     def __init__(self, parent=None):
84         super().__init__(parent)
85         self.setFixedSize(1550, 1080)
86         self.setMouseTracking(True)
87
88         self.drawing_mode = "continent"
89         self.current_continent_points = []
90         self.continents = []
91         self.continent_paths = []
92
93         self.plates = []
94         self.current_plate_points = None
95         self.current_plate_direction = (1, 0)
96         self.current_plate_speed = 1.0
97         self.current_plate_direction_name = "Восток"

```

```

97
98     self.height_map = None
99     self.humidity_map = None
100    self.temperature_map = None
101    self.biome_map = None
102    self.wind_map = None
103
104    self.sea_level = 0.3
105    self.show_biomes = False
106    self.show_heightmap = False
107    self.show_humidity = False
108    self.show_winds = False
109
110    self.pan_active = False
111    self.pan_start = QPoint()
112
113    def set_mode(self, mode):
114        self.drawing_mode = mode
115        if mode == "continent":
116            self.current_continent_points = []
117        else:
118            self.current_plate_points = []
119        self.update()
120
121    def set_plate_direction(self, dx, dy, name):
122        self.current_plate_direction = (dx, dy)
123        self.current_plate_direction_name = name
124
125    def set_plate_speed(self, speed):
126        self.current_plate_speed = speed
127
128    def clear_all(self):
129        self.current_continent_points = []
130        self.continents = []
131        self.continent_paths = []
132        self.plates = []
133        self.current_plate_points = None
134        self.height_map = None
135        self.humidity_map = None
136        self.temperature_map = None
137        self.biome_map = None
138        self.wind_map = None
139        self.update()
140
141    def mousePressEvent(self, event):
142        if event.button() == Qt.LeftButton:
143            pos = event.pos()
144            if self.drawing_mode == "continent":
145                self.current_continent_points = [pos]
146            else:
147                self.current_plate_points = [pos]
148            self.update()

```

```

149         elif event.button() == Qt.RightButton:
150             self.pan_active = True
151             self.pan_start = event.pos()
152             self.setCursor(Qt.ClosedHandCursor)
153
154     def mouseMoveEvent(self, event):
155         if event.buttons() & Qt.LeftButton:
156             pos = event.pos()
157             if self.drawing_mode == "continent" and self.
current_continent_points:
158                 self.current_continent_points.append(pos)
159                 self.update()
160             elif self.drawing_mode == "plate" and self.current_plate_points is
not None:
161                 self.current_plate_points.append(pos)
162                 self.update()
163             elif event.buttons() & Qt.RightButton and self.pan_active:
164                 delta = event.pos() - self.pan_start
165                 scroll_area = self.parent().parent()
166                 if isinstance(scroll_area, QScrollArea):
167                     h_bar = scroll_area.horizontalScrollBar()
168                     v_bar = scroll_area.verticalScrollBar()
169                     h_bar.setValue(h_bar.value() - delta.x())
170                     v_bar.setValue(v_bar.value() - delta.y())
171                 self.pan_start = event.pos()
172
173     def mouseReleaseEvent(self, event):
174         if event.button() == Qt.LeftButton:
175             if self.drawing_mode == "continent" and self.
current_continent_points:
176                 if len(self.current_continent_points) > 2:
177                     self.current_continent_points.append(self.
current_continent_points[0])
178                     self.update()
179             elif self.drawing_mode == "plate" and self.current_plate_points
and len(self.current_plate_points) > 2:
180                 self.current_plate_points.append(self.current_plate_points[0])
181                 plate = Plate(self.current_plate_points[:],
182                             self.current_plate_direction,
183                             self.current_plate_speed,
184                             self.current_plate_direction_name)
185                 self.plates.append(plate)
186                 self.current_plate_points = None
187                 self.update()
188             elif event.button() == Qt.RightButton:
189                 self.pan_active = False
190                 self.setCursor(Qt.ArrowCursor)
191
192     def min_distance_to_continents(self, new_poly):
193         w = self.width()
194         h = self.height()
195         min_dist = float('inf')

```

```

196         for cont in self.continents:
197             for dx in (-1, 0, 1):
198                 for dy in (-1, 0, 1):
199                     shifted = [QPointF(p.x() + dx * w, p.y() + dy * h) for p
in cont]
200                     for p1 in new_poly:
201                         for p2 in shifted:
202                             dist = math.hypot(p1.x() - p2.x(), p1.y() - p2.y())
203
204                             if dist < min_dist:
205                                 min_dist = dist
206
207         return min_dist
208
209 def paintEvent(self, event):
210     painter = QPainter(self)
211     painter.setRenderHint(QPainter.Antialiasing)
212     painter.fillRect(self.rect(), QColor(173, 216, 230))
213
214     if self.biome_map is not None and self.show_biomes:
215         self.draw_biome_map(painter)
216     elif self.humidity_map is not None and self.show_humidity:
217         self.draw_humidity_map(painter)
218     elif self.height_map is not None and self.show_heightmap:
219         self.draw_height_map(painter)
220     else:
221         w = self.width()
222         h = self.height()
223         for cont in self.continents:
224             painter.setBrush(QBrush(QColor(144, 238, 144)))
225             painter.setPen(QPen(Qt.darkGreen, 2))
226             painter.drawPolygon(QPolygonF(cont))
227
228         if self.drawing_mode == "continent" and len(self.
current_continent_points) > 1:
229             painter.setPen(QPen(Qt.gray, 2))
230             for i in range(len(self.current_continent_points)-1):
231                 painter.drawLine(self.current_continent_points[i], self.
current_continent_points[i+1])
232
233         for plate in self.plates:
234             painter.setPen(QPen(Qt.red, 2))
235             painter.setBrush(Qt.NoBrush)
236             if len(plate.points) > 1:
237                 painter.drawPolygon(QPolygonF(plate.points))
238             if plate.points:
239                 center = self.polygon_center(plate.points)
240                 dx, dy = plate.direction
241                 length = math.hypot(dx, dy)
242                 if length > 0:
243                     dx /= length
244                     dy /= length
245                     end = center + QPointF(dx * 40, dy * 40)

```

```

244         painter.setPen(QPen(Qt.red, 3))
245         painter.drawLine(center, end)
246         arrow_size = 12
247         angle = math.atan2(dy, dx)
248         p1 = end - QPointF(arrow_size * math.cos(angle + math.pi
/6), arrow_size * math.sin(angle + math.pi/6))
249         p2 = end - QPointF(arrow_size * math.cos(angle - math.pi
/6), arrow_size * math.sin(angle - math.pi/6))
250         painter.drawLine(end, p1)
251         painter.drawLine(end, p2)
252
253         if self.drawing_mode == "plate" and self.current_plate_points and
len(self.current_plate_points) > 1:
254             painter.setPen(QPen(QColor(255, 165, 0), 2))
255             for i in range(len(self.current_plate_points)-1):
256                 painter.drawLine(self.current_plate_points[i], self.
current_plate_points[i+1])
257
258         if self.wind_map is not None and self.show_winds:
259             self.draw_wind_map(painter)
260
261     def draw_wind_map(self, painter):
262         if self.wind_map is None:
263             return
264         h, w = self.wind_map.shape[:2]
265         step = 45
266
267         for y in range(step//2, h, step):
268             for x in range(step//2, w, step):
269                 dx, dy, speed = self.wind_map[y, x]
270                 if speed > 0.01:
271                     center = QPointF(x, y)
272                     arrow_length = 15 + speed * 25
273                     end = center + QPointF(dx * arrow_length, dy *
arrow_length)
274
275                     if speed > 0.7:
276                         painter.setPen(QPen(QColor(200, 50, 50), 2))
277                     elif speed > 0.4:
278                         painter.setPen(QPen(QColor(100, 100, 200), 2))
279                     else:
280                         painter.setPen(QPen(QColor(80, 80, 80), 1))
281
282                     painter.drawLine(center, end)
283
284                     angle = math.atan2(dy, dx)
285                     arrow_size = 8
286                     p1 = end - QPointF(arrow_size * math.cos(angle + math.pi
/6), arrow_size * math.sin(angle + math.pi/6))
287                     p2 = end - QPointF(arrow_size * math.cos(angle - math.pi
/6), arrow_size * math.sin(angle - math.pi/6))
288                     painter.drawLine(end, p1)

```

```

289         painter.drawLine(end, p2)
290
291     def draw_humidity_map(self, painter):
292         if self.humidity_map is None:
293             return
294         h, w = self.humidity_map.shape
295         block_size = 4
296
297         for y in range(0, h, block_size):
298             for x in range(0, w, block_size):
299                 humidity = self.humidity_map[y, x]
300
301                 if humidity < 0.2:
302                     color = QColor(160, 120, 60)
303                 elif humidity < 0.35:
304                     t = (humidity - 0.2) / 0.15
305                     r = int(160 + t * 50)
306                     g = int(120 + t * 50)
307                     b = 60
308                     color = QColor(r, g, b)
309                 elif humidity < 0.5:
310                     t = (humidity - 0.35) / 0.15
311                     r = int(210 - t * 30)
312                     g = int(170 + t * 40)
313                     b = 70
314                     color = QColor(r, g, b)
315                 elif humidity < 0.65:
316                     t = (humidity - 0.5) / 0.15
317                     r = int(180 - t * 100)
318                     g = int(210 - t * 30)
319                     b = 80
320                     color = QColor(r, g, b)
321                 elif humidity < 0.8:
322                     t = (humidity - 0.65) / 0.15
323                     r = int(80 - t * 40)
324                     g = int(180 - t * 0)
325                     b = 70
326                     color = QColor(r, g, b)
327                 else:
328                     color = QColor(40, 120, 40)
329
330                 painter.fillRect(QRect(x, y, block_size, block_size), color)
331
332     def draw_height_map(self, painter):
333         if self.height_map is None:
334             return
335         h, w = self.height_map.shape
336         block_size = 8
337         for y in range(0, h, block_size):
338             for x in range(0, w, block_size):
339                 height = self.height_map[y, x]
340                 if height < self.sea_level:

```

```

341         t = height / self.sea_level
342         intensity = 50 + int(t * 105)
343         color = QColor(20, 40, 80 + intensity)
344     else:
345         normalized = (height - self.sea_level) / (1 - self.
sea_level)
346         if normalized < 0.15:
347             green = 120 + int(normalized * 100)
348             color = QColor(60, green, 40)
349         elif normalized < 0.4:
350             t2 = (normalized - 0.15) / 0.25
351             green = 140 - int(t2 * 30)
352             brown = 60 + int(t2 * 70)
353             color = QColor(brown, green, 30)
354         elif normalized < 0.7:
355             t3 = (normalized - 0.4) / 0.3
356             brown = 100 + int(t3 * 70)
357             color = QColor(brown, int(brown * 0.7), int(brown *
0.4))
358         elif normalized < 0.9:
359             t4 = (normalized - 0.7) / 0.2
360             gray = 120 + int(t4 * 50)
361             color = QColor(gray, gray, gray)
362         else:
363             t5 = (normalized - 0.9) / 0.1
364             white_intensity = 200 + int(t5 * 55)
365             color = QColor(white_intensity, white_intensity,
white_intensity)
366         painter.fillRect(QRect(x, y, block_size, block_size), color)
367
368     def draw_biome_map(self, painter):
369         if self.biome_map is None:
370             return
371         biome_colors = {
372             BiomeType.TROPICAL_RAINFOREST: QColor(45, 90, 39),
373             BiomeType.FOREST: QColor(60, 158, 45),
374             BiomeType.TAIGA: QColor(44, 110, 42),
375             BiomeType.TUNDRA: QColor(139, 166, 169),
376             BiomeType.DESERT: QColor(240, 213, 140),
377             BiomeType.SAVANNA: QColor(194, 178, 120),
378             BiomeType.STEPPE: QColor(166, 160, 94),
379             BiomeType.OCEAN: QColor(26, 61, 110),
380             BiomeType.SEA: QColor(42, 93, 142),
381             BiomeType.MOUNTAIN: QColor(139, 140, 122)
382         }
383         h, w = self.biome_map.shape
384         block_size = 8
385         for y in range(0, h, block_size):
386             for x in range(0, w, block_size):
387                 biome = self.biome_map[y, x]
388                 if biome is not None:

```



```

389         painter.fillRect(QRect(x, y, block_size, block_size),
biome_colors.get(biome, QColor(255,255,255)))
390
391     def polygon_center(self, points):
392         if not points:
393             return QPointF(0,0)
394         x = sum(p.x() for p in points) / len(points)
395         y = sum(p.y() for p in points) / len(points)
396         return QPointF(x, y)
397
398     def add_continent(self, polygon):
399         if not polygon or len(polygon) < 3:
400             return False
401         if self.min_distance_to_continents(polygon) < 20:
402             return False
403         self.continents.append(polygon)
404         self.continent_paths = [polygon_to_path(cont) for cont in self.
continents]
405         self.update()
406         return True
407
408     def is_point_on_land(self, x, y, w, h):
409         for path in self.continent_paths:
410             if path is None:
411                 continue
412             for dx in (-1, 0, 1):
413                 for dy in (-1, 0, 1):
414                     px = x + dx * w
415                     py = y + dy * h
416                     if path.contains(QPointF(px, py)):
417                         return True
418         return False
419
420     def set_height_map(self, hmap):
421         self.height_map = hmap
422         self.show_heightmap = True
423         self.show_biomes = False
424         self.show_humidity = False
425         self.show_winds = False
426         self.update()
427
428     def set_humidity_map(self, hmap, temp_map, wind_map):
429         self.humidity_map = hmap
430         self.temperature_map = temp_map
431         self.wind_map = wind_map
432         self.show_humidity = True
433         self.show_heightmap = False
434         self.show_biomes = False
435         self.update()
436
437     def set_biome_map(self, bmap):
438         self.biome_map = bmap

```

```

439         self.show_biomes = True
440         self.show_heightmap = False
441         self.show_humidity = False
442         self.show_winds = False
443         self.update()
444
445 # ----- Главное окно -----
446 class TerrainGeneratorApp(QMainWindow):
447     def __init__(self):
448         super().__init__()
449         self.setWindowTitle("Генератор карт местности")
450         self.setFixedSize(1400, 800)
451
452         self.roughness = 0.5
453         self.iterations = 5
454         self.epsilon = 5
455         self.mountain_roughness = 0.5
456         self.plate_direction = (1, 0)
457         self.plate_speed = 1.0
458         self.plate_direction_name = "Восток"
459         self.global_temp = 25
460         self.sea_level = 0.3
461
462         self.trade_wind_speed = 0.8
463         self.westerly_speed = 0.6
464         self.polar_speed = 0.4
465
466         central = QWidget()
467         self.setCentralWidget(central)
468         main_layout = QHBoxLayout(central)
469         main_layout.setContentsMargins(5,5,5,5)
470
471         self.scroll_area = QScrollArea()
472         self.scroll_area.setWidgetResizable(False)
473         self.scroll_area.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOn)
474         self.scroll_area.setVerticalScrollBarPolicy(Qt.ScrollBarAlwaysOn)
475         self.canvas = MapCanvas()
476         self.scroll_area.setWidget(self.canvas)
477         main_layout.addWidget(self.scroll_area, stretch=3)
478
479         right_scroll = QScrollArea()
480         right_scroll.setWidgetResizable(True)
481         right_scroll.setFixedWidth(350)
482         main_layout.addWidget(right_scroll)
483
484         control_panel = QWidget()
485         right_scroll.setWidget(control_panel)
486         control_layout = QVBoxLayout(control_panel)
487         control_layout.setSpacing(10)
488
489         mode_group = QGroupBox("Режим работы")
490         mode_layout = QVBoxLayout()

```

```

491     btn_continent = QPushButton("Рисовать материк")
492     btn_plate = QPushButton("Рисовать плиту")
493     btn_continent.clicked.connect(lambda: self.canvas.set_mode("continent"
))
494     btn_plate.clicked.connect(lambda: self.canvas.set_mode("plate"))
495     mode_layout.addWidget(btn_continent)
496     mode_layout.addWidget(btn_plate)
497     mode_group.setLayout(mode_layout)
498     control_layout.addWidget(mode_group)
499
500     plate_group = QGroupBox("Движение плиты")
501     plate_layout = QVBoxLayout()
502     plate_layout.addWidget(QLabel("Направление:"))
503     dir_grid = QGridLayout()
504     directions = [
505         (0, -1, "Север", 0, 1),
506         (1, -1, "Северовосток-", 0, 2),
507         (1, 0, "Восток", 1, 2),
508         (1, 1, "Юговосток-", 2, 2),
509         (0, 1, "Юг", 2, 1),
510         (-1, 1, "Югозапад-", 2, 0),
511         (-1, 0, "Запад", 1, 0),
512         (-1, -1, "Северозапад-", 0, 0)
513     ]
514     for dx, dy, name, row, col in directions:
515         btn = QPushButton(name)
516         btn.clicked.connect(lambda checked, dx=dx, dy=dy, name=name: self.
set_plate_direction(dx, dy, name))
517         dir_grid.addWidget(btn, row, col)
518     plate_layout.addLayout(dir_grid)
519
520     plate_layout.addWidget(QLabel("Скорость плит влияет( на высоту гор!):"
))
521     self.speed_slider = QSlider(Qt.Horizontal)
522     self.speed_slider.setRange(5, 30)
523     self.speed_slider.setValue(10)
524     self.speed_slider.valueChanged.connect(self.on_speed_changed)
525     plate_layout.addWidget(self.speed_slider)
526     self.speed_label = QLabel("1.0")
527     plate_layout.addWidget(self.speed_label)
528     plate_group.setLayout(plate_layout)
529     control_layout.addWidget(plate_group)
530
531     wind_group = QGroupBox("Настройки ветров влияют( на влажность!)"
)
532     wind_layout = QVBoxLayout()
533
534     wind_layout.addWidget(QLabel("Пассаты тропики():"))
535     self.trade_slider = QSlider(Qt.Horizontal)
536     self.trade_slider.setRange(0, 100)
537     self.trade_slider.setValue(80)
538     self.trade_slider.valueChanged.connect(self.on_trade_wind_changed)
539     wind_layout.addWidget(self.trade_slider)

```

```

540 self.trade_label = QLabel("80%")
541 wind_layout.addWidget(self.trade_label)
542
543 wind_layout.addWidget(QLabel("Западные ветры умеренные():"))
544 self.westerly_slider = QSlider(Qt.Horizontal)
545 self.westerly_slider.setRange(0, 100)
546 self.westerly_slider.setValue(60)
547 self.westerly_slider.valueChanged.connect(self.on_westerly_changed)
548 wind_layout.addWidget(self.westerly_slider)
549 self.westerly_label = QLabel("60%")
550 wind_layout.addWidget(self.westerly_label)
551
552 wind_layout.addWidget(QLabel("Полярные ветры:"))
553 self.polar_slider = QSlider(Qt.Horizontal)
554 self.polar_slider.setRange(0, 100)
555 self.polar_slider.setValue(40)
556 self.polar_slider.valueChanged.connect(self.on_polar_changed)
557 wind_layout.addWidget(self.polar_slider)
558 self.polar_label = QLabel("40%")
559 wind_layout.addWidget(self.polar_label)
560
561 wind_group.setLayout(wind_layout)
562 control_layout.addWidget(wind_group)
563
564 params_group = QGroupBox("Параметры")
565 params_layout = QVBoxLayout()
566 params_layout.addWidget(QLabel("Шероховатость:"))
567 self.roughness_slider = QSlider(Qt.Horizontal)
568 self.roughness_slider.setRange(10, 100)
569 self.roughness_slider.setValue(50)
570 self.roughness_slider.valueChanged.connect(lambda v: setattr(self, '
roughness', v/100))
571 params_layout.addWidget(self.roughness_slider)
572
573 params_layout.addWidget(QLabel("Глубина рекурсии:"))
574 self.iter_spin = QSpinBox()
575 self.iter_spin.setRange(1, 8)
576 self.iter_spin.setValue(5)
577 self.iter_spin.valueChanged.connect(lambda v: setattr(self, '
iterations', v))
578 params_layout.addWidget(self.iter_spin)
579
580 params_layout.addWidget(QLabel("Уровень моря:"))
581 self.sea_slider = QSlider(Qt.Horizontal)
582 self.sea_slider.setRange(10, 70)
583 self.sea_slider.setValue(30)
584 self.sea_slider.valueChanged.connect(self.on_sea_level_changed)
585 params_layout.addWidget(self.sea_slider)
586
587 params_layout.addWidget(QLabel("Глобальная температура влияет( на
биомы!):"))
588 self.temp_slider = QSlider(Qt.Horizontal)

```

```

589     self.temp_slider.setRange(-20, 40)
590     self.temp_slider.setValue(25)
591     self.temp_slider.valueChanged.connect(self.on_global_temp_changed)
592     params_layout.addWidget(self.temp_slider)
593     self.temp_label = QLabel("°25C")
594     params_layout.addWidget(self.temp_label)
595
596     params_group.setLayout(params_layout)
597     control_layout.addWidget(params_group)
598
599     actions_group = QGroupBox("Действия")
600     actions_layout = QVBoxLayout()
601     btn_clear = QPushButton("Очистить всё")
602     btn_gen_cont = QPushButton("1. Сгенерировать материк")
603     btn_gen_height = QPushButton("2. Создать карту высот")
604     btn_gen_humidity = QPushButton("3. Создать карту влажности")
605     btn_gen_biomes = QPushButton("4. Создать биомы")
606     btn_show_full = QPushButton("Показать биомы")
607     btn_show_humidity = QPushButton("Показать влажность")
608     btn_show_height = QPushButton("Показать высоты")
609     btn_show_winds = QPushButton("Показать ветра")
610     btn_save = QPushButton("Сохранить карту")
611     btn_load = QPushButton("Загрузить карту")
612
613     btn_clear.clicked.connect(self.clear_all)
614     btn_gen_cont.clicked.connect(self.generate_continent)
615     btn_gen_height.clicked.connect(self.generate_heightmap)
616     btn_gen_humidity.clicked.connect(self.generate_humidity_map)
617     btn_gen_biomes.clicked.connect(self.generate_biomes)
618     btn_show_full.clicked.connect(self.show_full_map)
619     btn_show_humidity.clicked.connect(self.show_humidity_map)
620     btn_show_height.clicked.connect(self.show_height_map)
621     btn_show_winds.clicked.connect(self.show_winds)
622     btn_save.clicked.connect(self.save_map)
623     btn_load.clicked.connect(self.load_map)
624
625     actions_layout.addWidget(btn_clear)
626     actions_layout.addWidget(btn_gen_cont)
627     actions_layout.addWidget(btn_gen_height)
628     actions_layout.addWidget(btn_gen_humidity)
629     actions_layout.addWidget(btn_gen_biomes)
630     actions_layout.addWidget(btn_show_full)
631     actions_layout.addWidget(btn_show_humidity)
632     actions_layout.addWidget(btn_show_height)
633     actions_layout.addWidget(btn_show_winds)
634     actions_layout.addWidget(btn_save)
635     actions_layout.addWidget(btn_load)
636     actions_group.setLayout(actions_layout)
637     control_layout.addWidget(actions_group)
638
639     info_group = QGroupBox("Инструкция")
640     info_text = QLabel(

```

```

641         "1. Нарисуйте материк\n"
642         "2. Нажмите Сгенерировать ' материк '\n"
643         "3. Нарисуйте 2+ литосферные плиты\n"
644         "4. Настройте СКОРОСТЬ ПЛИТ влияет( на высоту гор!)\n"
645         "5. Нажмите Создать ' карту высот '\n"
646         "6. Настройте скорость ветров и температуру\n"
647         "7. Нажмите Создать ' карту влажности ' ветра( влияют!)\n"
648         "8. Нажмите Создать ' биомы '\n\n"
649         "СОХРАНЕНИЕ: сохраняет все карты и настройки в JSON\n"
650         "ЗАГРУЗКА: загружает ранее сохранённую карту\n\n"
651         "ПКМ + перетаскивание - перемещение"
652     )
653     info_text.setWordWrap(True)
654     info_layout = QVBoxLayout()
655     info_layout.addWidget(info_text)
656     info_group.setLayout(info_layout)
657     control_layout.addWidget(info_group)
658
659     self.status_label = QLabel("Готов. Режим: рисование материка")
660     self.status_label.setFrameStyle(QFrame.Sunken)
661     control_layout.addWidget(self.status_label)
662     control_layout.addStretch()
663
664     # ----- ФУНКЦИЯ КОЭФФИЦИЕНТА ИСКАЖЕНИЯ ПО ШИРОТЕ
665     -----
666     def get_distortion_factor(self, y, h):
667         center_y = h / 2
668         lat_norm = abs(y - center_y) / center_y
669         distortion = 1.0 - lat_norm * 0.6
670         return max(0.4, min(1.0, distortion))
671
672     # ----- СОХРАНЕНИЕ И ЗАГРУЗКА
673     -----
674     def save_map(self):
675         """Сохраняет всю карту в JSON файл"""
676         if self.canvas.height_map is None:
677             QMessageBox.warning(self, "Предупреждение", "Нет созданной карты
678             для сохранения!")
679             return
680
681         file_path, _ = QFileDialog.getSaveFileName(self, "Сохранить карту", ""
682         , "JSON files (*.json);;All Files (*)")
683         if not file_path:
684             return
685
686         # Добавляем расширение .json если его нет
687         if not file_path.endswith('.json'):
688             file_path += '.json'
689
690     try:
691         # Подготовка данных для сохранения

```

```

689         save_data = {
690             'version': '1.0',
691             'settings': {
692                 'roughness': self.roughness,
693                 'iterations': self.iterations,
694                 'epsilon': self.epsilon,
695                 'mountain_roughness': self.mountain_roughness,
696                 'plate_speed': self.plate_speed,
697                 'global_temp': self.global_temp,
698                 'sea_level': self.sea_level,
699                 'trade_wind_speed': self.trade_wind_speed,
700                 'westerly_speed': self.westerly_speed,
701                 'polar_speed': self.polar_speed
702             },
703             'continents': [(p.x(), p.y()) for p in cont] for cont in self
.canvas.continents],
704             'plates': [plate.to_dict() for plate in self.canvas.plates],
705         }
706
707         # Сохранение карты высот
708         if self.canvas.height_map is not None:
709             save_data['height_map'] = self.canvas.height_map.tolist()
710
711         # Сохранение карты влажности
712         if self.canvas.humidity_map is not None:
713             save_data['humidity_map'] = self.canvas.humidity_map.tolist()
714
715         # Сохранение карты биомов
716         if self.canvas.biome_map is not None:
717             biome_map_data = [[b.value if b is not None else None for b in
row] for row in self.canvas.biome_map]
718             save_data['biome_map'] = biome_map_data
719
720         # Сохранение карты температур
721         if self.canvas.temperature_map is not None:
722             save_data['temperature_map'] = self.canvas.temperature_map.
tolist()
723
724         # Сохранение карты ветров
725         if self.canvas.wind_map is not None:
726             save_data['wind_map'] = self.canvas.wind_map.tolist()
727
728         # Запись в файл
729         with open(file_path, 'w', encoding='utf-8') as f:
730             json.dump(save_data, f, ensure_ascii=False, indent=2)
731
732         QMessageBox.information(self, "Сохранение", f"Карта успешно
сохранена в {file_path}")
733         self.status_label.setText(f"Карта сохранена: {file_path}")
734
735     except Exception as e:

```



```

736         QMessageBox.critical(self, "Ошибка", f"Не удалось сохранить
карту:\n{str(e)}")
737
738     def load_map(self):
739         """Загружает карту из JSON файла"""
740         file_path, _ = QFileDialog.getOpenFileName(self, "Загрузить карту", ""
, "JSON files (*.json);;All Files (*)")
741         if not file_path:
742             return
743
744         try:
745             with open(file_path, 'r', encoding='utf-8') as f:
746                 load_data = json.load(f)
747
748             # Очистка текущей карты
749             self.clear_all()
750
751             # Загрузка настроек
752             settings = load_data.get('settings', {})
753             self.roughness = settings.get('roughness', self.roughness)
754             self.iterations = settings.get('iterations', self.iterations)
755             self.epsilon = settings.get('epsilon', self.epsilon)
756             self.mountain_roughness = settings.get('mountain_roughness', self.
mountain_roughness)
757             self.plate_speed = settings.get('plate_speed', self.plate_speed)
758             self.global_temp = settings.get('global_temp', self.global_temp)
759             self.sea_level = settings.get('sea_level', self.sea_level)
760             self.trade_wind_speed = settings.get('trade_wind_speed', self.
trade_wind_speed)
761             self.westerly_speed = settings.get('westerly_speed', self.
westerly_speed)
762             self.polar_speed = settings.get('polar_speed', self.polar_speed)
763
764             # Обновление UI элементов
765             self.roughness_slider.setValue(int(self.roughness * 100))
766             self.iter_spin.setValue(self.iterations)
767             self.speed_slider.setValue(int(self.plate_speed * 10))
768             self.speed_label.setText(f"{self.plate_speed:.1f}")
769             self.temp_slider.setValue(self.global_temp)
770             self.temp_label.setText(f"{self.global_temp}°C")
771             self.sea_slider.setValue(int(self.sea_level * 100))
772             self.trade_slider.setValue(int(self.trade_wind_speed * 100))
773             self.westerly_slider.setValue(int(self.westerly_speed * 100))
774             self.polar_slider.setValue(int(self.polar_speed * 100))
775
776             # Загрузка материков
777             continents_data = load_data.get('continents', [])
778             for cont_data in continents_data:
779                 polygon = [QPointF(x, y) for x, y in cont_data]
780                 self.canvas.continents.append(polygon)
781                 self.canvas.continent_paths = [polygon_to_path(cont) for cont in
self.canvas.continents]

```



```

782
783     # Загрузка плит
784     plates_data = load_data.get('plates', [])
785     for plate_data in plates_data:
786         plate = Plate.from_dict(plate_data)
787         self.canvas.plates.append(plate)
788
789     # Загрузка карты высот
790     if 'height_map' in load_data:
791         height_data = np.array(load_data['height_map'], dtype=np.
float32)
792         self.canvas.height_map = height_data
793         self.canvas.show_heightmap = True
794
795     # Загрузка карты влажности
796     if 'humidity_map' in load_data:
797         humidity_data = np.array(load_data['humidity_map'], dtype=np.
float32)
798         self.humidity_map = humidity_data
799         self.canvas.humidity_map = humidity_data
800
801     # Загрузка карты биомов
802     if 'biome_map' in load_data:
803         biome_data = load_data['biome_map']
804         biome_map = np.zeros((len(biome_data), len(biome_data[0]) if
biome_data else 0), dtype=object)
805         for y, row in enumerate(biome_data):
806             for x, val in enumerate(row):
807                 if val is not None:
808                     biome_map[y, x] = BiomeType.from_dict(val)
809         self.canvas.biome_map = biome_map
810
811     # Загрузка карты температур
812     if 'temperature_map' in load_data:
813         self.canvas.temperature_map = np.array(load_data['
temperature_map'], dtype=np.float32)
814
815     # Загрузка карты ветров
816     if 'wind_map' in load_data:
817         wind_data = load_data['wind_map']
818         self.canvas.wind_map = np.array(wind_data, dtype=np.float32)
819
820     # Обновление уровня моря на холсте
821     self.canvas.sea_level = self.sea_level
822
823     # Перерисовка
824     self.canvas.update()
825
826     QMessageBox.information(self, "Загрузка", f"Карта успешно
загружена из {file_path}")
827     self.status_label.setText(f"Карта загружена: {file_path}")
828

```

```

829         except Exception as e:
830             QMessageBox.critical(self, "Ошибка", f"Не удалось загрузить
карту:\n{str(e)}")
831
832     def on_trade_wind_changed(self, value):
833         self.trade_wind_speed = value / 100
834         self.trade_label.setText(f"{value}%")
835         self.status_label.setText(f"Пассаты: {value}% -> влияет на влажность
выше( = влажнее)")
836
837     def on_westerly_changed(self, value):
838         self.westerly_speed = value / 100
839         self.westerly_label.setText(f"{value}%")
840         self.status_label.setText(f"Западные ветры: {value}% -> влияет на
влажность выше( = влажнее)")
841
842     def on_polar_changed(self, value):
843         self.polar_speed = value / 100
844         self.polar_label.setText(f"{value}%")
845         self.status_label.setText(f"Полярные ветры: {value}% -> влияет на
влажность выше( = влажнее)")
846
847     def on_global_temp_changed(self, value):
848         self.global_temp = value
849         self.temp_label.setText(f"{value}°C")
850         self.status_label.setText(f"Температура: {value}°C нажмите( Создать '
биомы' для применения)")
851
852     def set_plate_direction(self, dx, dy, name):
853         self.plate_direction = (dx, dy)
854         self.plate_direction_name = name
855         self.canvas.set_plate_direction(dx, dy, name)
856         self.status_label.setText(f"Направление плиты: {name}")
857
858     def on_speed_changed(self, value):
859         self.plate_speed = value / 10.0
860         self.speed_label.setText(f"{self.plate_speed:.1f}")
861         self.canvas.set_plate_speed(self.plate_speed)
862         self.status_label.setText(f"Скорость плит: {self.plate_speed:.1f}
выше( скорость = выше горы)")
863
864     def on_sea_level_changed(self, value):
865         self.sea_level = value / 100.0
866         self.canvas.sea_level = self.sea_level
867         if self.canvas.height_map is not None:
868             self.canvas.show_heightmap = True
869             self.canvas.show_biomes = False
870             self.canvas.show_humidity = False
871             self.canvas.update()
872
873     def clear_all(self):
874         self.canvas.clear_all()

```

```

875         self.status_label.setText("Очищено. Режим: рисование материка")
876
877     def generate_continent(self):
878         if len(self.canvas.current_continent_points) < 3:
879             QMessageBox.warning(self, "Предупреждение", "Сначала нарисуйте
материк!")
880             return
881         points = [(p.x(), p.y()) for p in self.canvas.current_continent_points
]
882         simplified = self.douglas_peucker(points, self.epsilon)
883         if len(simplified) < 3:
884             simplified = points
885         fractal = []
886         for i in range(len(simplified)-1):
887             p1 = simplified[i]
888             p2 = simplified[i+1]
889             seg = self.displace_segment(p1, p2, self.roughness, self.
iterations)
890             if i == 0:
891                 fractal.extend(seg)
892             else:
893                 fractal.extend(seg[1:])
894         if len(fractal) < 3:
895             QMessageBox.warning(self, "Ошибка", "Не удалось создать
фрактальную линию.")
896             return
897         poly = [QPointF(x, y) for x, y in fractal]
898         if not self.canvas.add_continent(poly):
899             QMessageBox.warning(self, "Слишком близко", "Новый материк
слишком близко к существующему.")
900         else:
901             self.canvas.current_continent_points = []
902             self.status_label.setText(f"Материк добавлен. Всего: {len(self.
canvas.continents)}")
903
904         # ----- ГЕНЕРАЦИЯ ВЫСОТ
905         -----
906     def chaotic_noise(self, x, y, scale, seed=0):
907         n1 = math.sin(x * scale * 0.05 + seed) * math.cos(y * scale * 0.05 +
seed * 1.3)
908         n2 = math.sin(x * scale * 0.12 + seed * 2) * 0.7
909         n3 = math.cos(y * scale * 0.15 + seed * 1.7) * 0.7
910         n4 = math.sin((x + y) * scale * 0.08) * math.cos((x - y) * scale *
0.08)
911         return (n1 + n2 + n3 + n4) / 3.5
912
913     def generate_heightmap(self):
914         if len(self.canvas.continents) == 0:
915             QMessageBox.warning(self, "Предупреждение", "Сначала нарисуйте
хотя бы один материк!")
916             return
917         if len(self.canvas.plates) < 2:

```

```

917         QMessageBox.warning(self, "Предупреждение", "Нарисуйте хотя бы
две литосферные плиты!")
918         return
919
920         w = self.canvas.width()
921         h = self.canvas.height()
922
923         step = 8
924         cols = w // step + 1
925         rows = h // step + 1
926
927         random.seed(42)
928         np.random.seed(42)
929
930         self.status_label.setText("Генерация карты высот...")
931         QApplication.processEvents()
932
933         height_map = np.zeros((rows, cols))
934
935         for y in range(rows):
936             world_y = y * step
937             distortion = self.get_distortion_factor(world_y, h)
938
939             for x in range(cols):
940                 world_x = x * step
941                 if self.canvas.is_point_on_land(world_x, world_y, w, h):
942                     noise = self.perlin_noise(world_x / 150, world_y / 150)
943                     chaotic = self.chaotic_noise(world_x, world_y, 0.5) * 0.1
944                     height_map[y, x] = (0.35 + noise * 0.15 + chaotic) *
distortion
945                 else:
946                     height_map[y, x] = 0.1 * distortion
947
948         self.status_label.setText("Генерация карты высот... обработка
тектоники")
949         QApplication.processEvents()
950
951         plate_edges = []
952         for plate in self.canvas.plates:
953             edges = []
954             points = plate.points
955             if len(points) < 3:
956                 continue
957             for i in range(len(points) - 1):
958                 p1 = points[i]
959                 p2 = points[i + 1]
960                 edges.append((p1, p2))
961             plate_edges.append((plate, edges))
962
963         mountain_influence = np.zeros((rows, cols))
964         trench_influence = np.zeros((rows, cols))
965

```

```

966         current_plate_speed = self.plate_speed
967
968     for y in range(rows):
969         world_y = y * step
970         distortion = self.get_distortion_factor(world_y, h)
971
972         for x in range(cols):
973             world_x = x * step
974             if height_map[y, x] < 0.2:
975                 continue
976
977             for i, (plate1, edges1) in enumerate(plate_edges):
978                 for j, (plate2, edges2) in enumerate(plate_edges):
979                     if i >= j:
980                         continue
981
982                     dir1 = plate1.direction
983                     dir2 = plate2.direction
984                     dot = dir1[0] * dir2[0] + dir1[1] * dir2[1]
985                     approach = (1 - dot) / 2
986                     divergence = (dot + 1) / 2
987
988                     speed_factor = (plate1.speed + plate2.speed) / 2
989                     base_speed = max(0.3, min(2.0, speed_factor))
990
991                     dist_threshold = 30 + base_speed * 25
992
993                     min_dist1 = self.min_distance_to_edges(world_x,
994 world_y, edges1, w, h)
995                     min_dist2 = self.min_distance_to_edges(world_x,
996 world_y, edges2, w, h)
997
998                     if min_dist1 < dist_threshold and min_dist2 <
999 dist_threshold:
1000                         dist_factor = max(1 - min_dist1/dist_threshold, 1
1001 - min_dist2/dist_threshold)
1002                         height_multiplier = 0.4 + base_speed * 0.4
1003
1004                         if approach > 0.35:
1005                             intensity = approach * base_speed *
1006 dist_factor
1007                             mountain_influence[y, x] = max(
1008                                     min(0.85,
1009 intensity * height_multiplier * 1.2 * distortion))
1010
1011                         if divergence > 0.35:
1012                             intensity = divergence * base_speed *
1013 dist_factor
1014                             trench_influence[y, x] = max(trench_influence[
1015 y, x],

```

```

1008                                                                 min(0.45,
intensity * height_multiplier * distortion))
1009
1010     mountain_influence = self.gaussian_blur(mountain_influence, 2 + int(
self.plate_speed * 2))
1011     trench_influence = self.gaussian_blur(trench_influence, 2)
1012
1013     for y in range(rows):
1014         world_y = y * step
1015         distortion = self.get_distortion_factor(world_y, h)
1016
1017         for x in range(cols):
1018             if mountain_influence[y, x] > 0.05:
1019                 new_height = 0.45 + mountain_influence[y, x] * (0.5 + self
.plate_speed * 0.2)
1020                 height_map[y, x] = max(height_map[y, x], new_height *
distortion)
1021             if trench_influence[y, x] > 0.05:
1022                 height_map[y, x] = max(0.1, height_map[y, x] -
trench_influence[y, x] * 0.35)
1023
1024     for y in range(rows):
1025         world_y = y * step
1026         distortion = self.get_distortion_factor(world_y, h)
1027
1028         for x in range(cols):
1029             if height_map[y, x] >= self.sea_level:
1030                 normalized = (height_map[y, x] - self.sea_level) / (1 -
self.sea_level)
1031                 enhanced = normalized ** 1.25
1032                 height_map[y, x] = self.sea_level + enhanced * (1 - self.
sea_level)
1033
1034     self.status_label.setText("Генерация карты высот... интерполяция")
1035     QApplication.processEvents()
1036
1037     smoothed_low = self.gaussian_blur(height_map, 2)
1038
1039     full_height_map = np.zeros((h, w))
1040     for y in range(h):
1041         src_y = y / step
1042         y0 = int(src_y)
1043         if y0 >= rows - 1:
1044             y0 = rows - 2
1045         fy = src_y - y0
1046         for x in range(w):
1047             src_x = x / step
1048             x0 = int(src_x)
1049             if x0 >= cols - 1:
1050                 x0 = cols - 2
1051             fx = src_x - x0
1052

```

```

1053         v00 = smoothed_low[y0, x0]
1054         v10 = smoothed_low[y0, x0 + 1]
1055         v01 = smoothed_low[y0 + 1, x0]
1056         v11 = smoothed_low[y0 + 1, x0 + 1]
1057
1058         v0 = v00 * (1 - fx) + v10 * fx
1059         v1 = v01 * (1 - fx) + v11 * fx
1060         full_height_map[y, x] = v0 * (1 - fy) + v1 * fy
1061
1062         full_height_map = self.gaussian_blur(full_height_map, 2)
1063         full_height_map = np.clip(full_height_map, 0.05, 0.98)
1064
1065         self.canvas.set_height_map(full_height_map)
1066         self.status_label.setText(f"Капра высот создана! Скорость плит: {self.
plate_speed:.1f}, макс. высота: {full_height_map.max():.2f}")
1067
1068     def min_distance_to_edges(self, x, y, edges, world_w, world_h):
1069         min_dist = float('inf')
1070         for p1, p2 in edges:
1071             for dx in (-1, 0, 1):
1072                 for dy in (-1, 0, 1):
1073                     x1 = p1.x() + dx * world_w
1074                     y1 = p1.y() + dy * world_h
1075                     x2 = p2.x() + dx * world_w
1076                     y2 = p2.y() + dy * world_h
1077                     dist = self.point_to_segment_distance(x, y, x1, y1, x2, y2
)
1078                 if dist < min_dist:
1079                     min_dist = dist
1080         return min_dist
1081
1082     def gaussian_blur(self, array, radius):
1083         if radius < 1:
1084             return array
1085         result = array.copy()
1086         h, w = array.shape
1087         for _ in range(radius):
1088             temp = result.copy()
1089             for y in range(1, h - 1):
1090                 for x in range(1, w - 1):
1091                     temp[y, x] = (result[y-1, x-1] + result[y-1, x] + result[y
-1, x+1] +
1092                                     result[y, x-1] + result[y, x] + result[y, x
+1] +
1093                                     result[y+1, x-1] + result[y+1, x] + result[y
+1, x+1]) / 9
1094             result = temp
1095         return result
1096
1097     # ----- ГЕНЕРАЦИЯ ВЛАЖНОСТИ
1098     -----

```

```

1099 def create_wind_map(self, w, h):
1100     wind = np.zeros((h, w, 3))
1101     center_y = h / 2
1102
1103     for y in range(h):
1104         lat = (y - center_y) / center_y
1105         abs_lat = abs(lat)
1106
1107         for x in range(w):
1108             if abs_lat < 0.35:
1109                 if lat > 0:
1110                     dx = -0.85
1111                     dy = 0.85
1112                     speed = self.trade_wind_speed
1113                 else:
1114                     dx = 0.85
1115                     dy = -0.85
1116                     speed = self.trade_wind_speed
1117             elif abs_lat < 0.7:
1118                 if lat > 0:
1119                     dx = -0.7
1120                     dy = 0.7
1121                     speed = self.westerly_speed
1122                 else:
1123                     dx = -0.7
1124                     dy = -0.7
1125                     speed = self.westerly_speed
1126             else:
1127                 if lat > 0:
1128                     dx = 0.6
1129                     dy = -0.8
1130                     speed = self.polar_speed
1131                 else:
1132                     dx = 0.6
1133                     dy = 0.8
1134                     speed = self.polar_speed
1135
1136             length = math.hypot(dx, dy)
1137             if length > 0:
1138                 dx /= length
1139                 dy /= length
1140
1141             wind[y, x] = [dx, dy, speed]
1142
1143     return wind
1144
1145 def create_temperature_map(self, w, h, height_map, sea_level):
1146     temp = np.zeros((h, w))
1147     center_y = h / 2
1148
1149     for y in range(h):
1150         distortion = self.get_distortion_factor(y, h)

```



```

1151         lat_factor = 1 - abs(y - center_y) / center_y
1152         lat_factor = lat_factor ** 1.2
1153
1154         for x in range(w):
1155             base_temp = self.global_temp * lat_factor * distortion
1156
1157             height = height_map[y, x]
1158             if height < sea_level:
1159                 height_factor = 1.0
1160             else:
1161                 normalized_height = (height - sea_level) / (1 - sea_level)
1162                 height_factor = 1 - normalized_height * 0.5
1163
1164             t = base_temp * height_factor
1165             temp[y, x] = np.clip(t, -30, 45)
1166
1167         return temp
1168
1169     def distance_to_water_smooth(self, height_map, sea_level):
1170         h, w = height_map.shape
1171         distance = np.full((h, w), 500.0)
1172
1173         water_pixels = []
1174         step = 8
1175         for y in range(0, h, step):
1176             for x in range(0, w, step):
1177                 if height_map[y, x] < sea_level:
1178                     water_pixels.append((x, y))
1179
1180         for y in range(0, h, step):
1181             for x in range(0, w, step):
1182                 if height_map[y, x] >= sea_level:
1183                     min_dist = 500
1184                     for wx, wy in water_pixels:
1185                         dist = math.hypot(x - wx, y - wy)
1186                         if dist < min_dist:
1187                             min_dist = dist
1188                     distance[y, x] = min_dist
1189
1190         result = np.zeros((h, w))
1191         for y in range(h):
1192             for x in range(w):
1193                 yy = (y // step) * step
1194                 xx = (x // step) * step
1195                 if yy >= h:
1196                     yy = h - step
1197                 if xx >= w:
1198                     xx = w - step
1199                 result[y, x] = distance[yy, xx]
1200
1201         result = self.gaussian_blur(result, 2)
1202         return result

```

```

1203
1204     def calculate_rain_shadow_with_speed(self, x, y, wind_map, height_map,
1205 sea_level):
1206         dx, dy, wind_speed = wind_map[y, x]
1207
1208         steps = 40
1209         path_has_mountain = False
1210         path_has_water = False
1211         mountain_height_factor = 0
1212
1213         for step in range(1, steps + 1):
1214             px = int(x - dx * step * 2)
1215             py = int(y - dy * step * 2)
1216
1217             if px < 0 or px >= height_map.shape[1] or py < 0 or py >=
1218 height_map.shape[0]:
1219                 break
1220
1221             height = height_map[py, px]
1222
1223             if height < sea_level:
1224                 path_has_water = True
1225
1226             if height > 0.65 and not path_has_mountain:
1227                 path_has_mountain = True
1228                 mountain_height_factor = min(1.0, (height - 0.65) / 0.3)
1229
1230             wind_boost = 0.3 + wind_speed * 1.2
1231
1232             if path_has_mountain:
1233                 if path_has_water:
1234                     return (0.8 + mountain_height_factor * 0.4) * wind_boost
1235                 else:
1236                     return max(0.15, (0.35 - mountain_height_factor * 0.15)) * (1
1237 - wind_speed * 0.15)
1238             elif path_has_water:
1239                 return 0.9 * wind_boost
1240             else:
1241                 return 0.6
1242
1243     def generate_humidity_map(self):
1244         if self.canvas.height_map is None:
1245             QMessageBox.warning(self, "Предупреждение", "Сначала создайте
1246 карту высот!")
1247             return
1248
1249         h, w = self.canvas.height_map.shape
1250         height_map = self.canvas.height_map
1251         sea_level = self.sea_level
1252
1253         self.status_label.setText("Создание карты ветров...")
1254         QApplication.processEvents()

```

```

1251
1252     wind_map = self.create_wind_map(w, h)
1253
1254     self.status_label.setText("Создание карты температур...")
1255     QApplication.processEvents()
1256
1257     temperature_map = self.create_temperature_map(w, h, height_map,
1258 sea_level)
1259
1260     self.status_label.setText("Вычисление расстояния до воды...")
1261     QApplication.processEvents()
1262
1263     water_distance = self.distance_to_water_smooth(height_map, sea_level)
1264
1265     self.status_label.setText("Расчёт влажности с( учётом широтного
1266 искажения)...")
1267     QApplication.processEvents()
1268
1269     humidity_map = np.zeros((h, w))
1270     step = 6
1271
1272     for y in range(0, h, step):
1273         distortion = self.get_distortion_factor(y, h)
1274
1275         for x in range(0, w, step):
1276             height = height_map[y, x]
1277
1278             if height < sea_level:
1279                 humidity = 0.95
1280             else:
1281                 dist = water_distance[y, x]
1282                 water_factor = math.exp(-dist / 60)
1283
1284                 temp = temperature_map[y, x]
1285                 temp_factor = 0.3 + (temp + 10) / 60 * 0.6
1286                 temp_factor = np.clip(temp_factor, 0.3, 0.95)
1287
1288                 height_normalized = (height - sea_level) / (1 - sea_level)
1289                 height_factor = 1 - height_normalized * 0.7
1290
1291                 rain_factor = self.calculate_rain_shadow_with_speed(x, y,
1292 wind_map, height_map, sea_level)
1293
1294                 wind_speed = wind_map[y, x][2]
1295                 wind_direct_boost = wind_speed * 0.4
1296
1297                 humidity = (water_factor * 0.35 +
1298                             temp_factor * 0.25 +
1299                             height_factor * 0.2 +
1300                             rain_factor * 0.15 +
1301                             wind_direct_boost * 0.05)

```

```

1300         humidity = humidity * distortion
1301         humidity = np.clip(humidity, 0.08, 0.98)
1302
1303         for dy in range(step):
1304             for dx in range(step):
1305                 ny, nx = y + dy, x + dx
1306                 if ny < h and nx < w:
1307                     humidity_map[ny, nx] = humidity
1308
1309         humidity_map = self.gaussian_blur(humidity_map, 2)
1310
1311         avg_wind = (self.trade_wind_speed + self.westerly_speed + self.
1312 polar_speed) / 3
1313         avg_humidity = np.mean(humidity_map[height_map >= sea_level]) if np.
1314 any(height_map >= sea_level) else 0
1315
1316         self.humidity_map = humidity_map
1317         self.canvas.set_humidity_map(humidity_map, temperature_map, wind_map)
1318         self.status_label.setText(f"Карта влажности создана! Искажение:
1319 экватор=1.0, полюса=0.4 | Ветра: пассаты={self.trade_wind_speed*100:.0f}%,
1320 зап.={self.westerly_speed*100:.0f}%, пол.={self.polar_speed*100:.0f}% |
1321 Срвлажность.: {avg_humidity*100:.0f}%")
1322
1323     def generate_biomes(self):
1324         if self.humidity_map is None:
1325             QMessageBox.warning(self, "Предупреждение", "Сначала создайте
1326 карту влажности!")
1327             return
1328
1329         h, w = self.humidity_map.shape
1330         height_map = self.canvas.height_map
1331         biome_map = np.zeros((h, w), dtype=object)
1332
1333         temperature_map = self.create_temperature_map(w, h, height_map, self.
1334 sea_level)
1335         self.canvas.temperature_map = temperature_map
1336
1337         for y in range(h):
1338             for x in range(w):
1339                 height = height_map[y, x]
1340
1341                 if height < self.sea_level:
1342                     if height < self.sea_level - 0.15:
1343                         biome_map[y, x] = BiomeType.OCEAN
1344                     else:
1345                         biome_map[y, x] = BiomeType.SEA
1346                 else:
1347                     temp = temperature_map[y, x]
1348                     humidity = self.humidity_map[y, x]
1349
1350                     if height > 0.75:
1351                         biome_map[y, x] = BiomeType.MOUNTAIN

```

```

1345         elif humidity > 0.7:
1346             if temp > 28:
1347                 biome_map[y, x] = BiomeType.TROPICAL_RAINFOREST
1348             elif temp > 15:
1349                 biome_map[y, x] = BiomeType.FOREST
1350             elif temp > 0:
1351                 biome_map[y, x] = BiomeType.TAIGA
1352             else:
1353                 biome_map[y, x] = BiomeType.TUNDRA
1354         elif humidity > 0.45:
1355             if temp > 30:
1356                 biome_map[y, x] = BiomeType.SAVANNA
1357             elif temp > 18:
1358                 biome_map[y, x] = BiomeType.STEPPE
1359             elif temp > 0:
1360                 biome_map[y, x] = BiomeType.TAIGA
1361             else:
1362                 biome_map[y, x] = BiomeType.TUNDRA
1363         else:
1364             if temp > 30:
1365                 biome_map[y, x] = BiomeType.DESERT
1366             elif temp > 20:
1367                 biome_map[y, x] = BiomeType.STEPPE
1368             elif temp > -5:
1369                 biome_map[y, x] = BiomeType.TUNDRA
1370             else:
1371                 biome_map[y, x] = BiomeType.TUNDRA
1372
1373         self.canvas.set_biome_map(biome_map)
1374         self.status_label.setText(f"Карта биомов создана! температура(: {self.
1375 global_temp°}C)")
1376
1377 # ----- ВСПОМОГАТЕЛЬНЫЕ МЕТОДЫ
1378 -----
1379 def douglas_peucker(self, points, epsilon):
1380     if len(points) < 3:
1381         return points[:]
1382     first, last = points[0], points[-1]
1383     max_dist = 0
1384     index = 0
1385     for i in range(1, len(points)-1):
1386         dist = self.point_line_distance(points[i], first, last)
1387         if dist > max_dist:
1388             max_dist = dist
1389             index = i
1390     if max_dist > epsilon:
1391         left = self.douglas_peucker(points[:index+1], epsilon)
1392         right = self.douglas_peucker(points[index:], epsilon)
1393         return left[:-1] + right
1394     else:
1395         return [first, last]

```

```

1395 def point_line_distance(self, point, line_start, line_end):
1396     x0, y0 = point
1397     x1, y1 = line_start
1398     x2, y2 = line_end
1399     if x1 == x2 and y1 == y2:
1400         return math.hypot(x0 - x1, y0 - y1)
1401     numerator = abs((x2 - x1) * (y1 - y0) - (x1 - x0) * (y2 - y1))
1402     denominator = math.hypot(x2 - x1, y2 - y1)
1403     return numerator / denominator
1404
1405 def displace_segment(self, p1, p2, roughness, iterations):
1406     if iterations <= 0 or roughness < 0.01:
1407         return [p1, p2]
1408     mid_x = (p1[0] + p2[0]) / 2
1409     mid_y = (p1[1] + p2[1]) / 2
1410     dx = p2[0] - p1[0]
1411     dy = p2[1] - p1[1]
1412     length = math.hypot(dx, dy)
1413     if length < 1e-6:
1414         return [p1, p2]
1415     perp_x = -dy / length
1416     perp_y = dx / length
1417     max_shift = length * roughness
1418     shift = random.uniform(-max_shift, max_shift)
1419     displaced_x = mid_x + perp_x * shift
1420     displaced_y = mid_y + perp_y * shift
1421     left = self.displace_segment(p1, (displaced_x, displaced_y), roughness
1422 * 0.6, iterations - 1)
1423     right = self.displace_segment((displaced_x, displaced_y), p2,
1424 roughness * 0.6, iterations - 1)
1425     return left[:-1] + right
1426
1427 def point_to_segment_distance(self, px, py, x1, y1, x2, y2):
1428     dx = x2 - x1
1429     dy = y2 - y1
1430     if dx == 0 and dy == 0:
1431         return math.hypot(px - x1, py - y1)
1432     t = ((px - x1) * dx + (py - y1) * dy) / (dx*dx + dy*dy)
1433     if t < 0:
1434         return math.hypot(px - x1, py - y1)
1435     elif t > 1:
1436         return math.hypot(px - x2, py - y2)
1437     else:
1438         proj_x = x1 + t * dx
1439         proj_y = y1 + t * dy
1440         return math.hypot(px - proj_x, py - proj_y)
1441
1442 def perlin_noise(self, x, y):
1443     return (math.sin(x * 0.1) * math.cos(y * 0.1) +
1444             math.sin(x * 0.3 + 1) * math.cos(y * 0.3 + 1) * 0.5 +
1445             math.sin(x * 0.7 + 2) * math.cos(y * 0.7 + 2) * 0.25) / 1.75 +
0.5

```

```

1444
1445 def show_full_map(self):
1446     if self.canvas.biome_map is not None:
1447         self.canvas.show_biomes = True
1448         self.canvas.show_heightmap = False
1449         self.canvas.show_humidity = False
1450         self.canvas.show_winds = False
1451         self.canvas.update()
1452     elif self.canvas.height_map is not None:
1453         self.show_height_map()
1454
1455 def show_humidity_map(self):
1456     if self.humidity_map is not None:
1457         self.canvas.show_humidity = True
1458         self.canvas.show_biomes = False
1459         self.canvas.show_heightmap = False
1460         self.canvas.show_winds = False
1461         self.canvas.update()
1462     else:
1463         QMessageBox.warning(self, "Предупреждение", "Сначала создайте
1464 карту влажности!")
1465
1466 def show_height_map(self):
1467     if self.canvas.height_map is not None:
1468         self.canvas.show_heightmap = True
1469         self.canvas.show_biomes = False
1470         self.canvas.show_humidity = False
1471         self.canvas.show_winds = False
1472         self.canvas.update()
1473     else:
1474         QMessageBox.warning(self, "Предупреждение", "Сначала создайте
1475 карту высот!")
1476
1477 def show_winds(self):
1478     if self.canvas.wind_map is not None:
1479         self.canvas.show_winds = not self.canvas.show_winds
1480         self.canvas.update()
1481     else:
1482         QMessageBox.warning(self, "Предупреждение", "Сначала создайте
1483 карту влажности!")
1484
1485 if __name__ == "__main__":
1486     app = QApplication(sys.argv)
1487     window = TerrainGeneratorApp()
1488     window.show()
1489     sys.exit(app.exec_())

```